thyme v0.1.0 · Typst >= 0.15.0

Pure-Typst static site generator with typed hypertagged HTML, Tadi Web .typ source transparency, zero-JS CSS i18n, XML sitemaps & feeds, and a monochrome blog preset.

core ssg	`#import "@preview/thyme:0.1.0": *`
blog preset	`#import blog: *` or `blog.site.with(...)`
scaffold	`typst init @preview/thyme:0.1.0 my-site`
dev / build	`typst watch --features html,bundle -f bundle site.typ dist`
license	`MIT` (`src/`) · `MIT-0` (`template/`, `examples/`)

1 Two-Layer Architecture & Pure-Typst Workflow . . . 1
2 Tadi Web `.typ` Sources & `sws.toml` Headers . . . 1
3 Public Surface . . . 2
3.1 Core — `#import "@preview/thyme:0.1.0": *` . . . 2
3.2 `blog` — `#import "@preview/thyme:0.1.0": blog` then `#import blog: *` . . . 2
4 Internal Modules (`src/**/*.typ`) . . . 4
4.1 Monochrome Theme & Fonts (`src/blog/theme.typ`) . . . 4

01 Two-Layer Architecture & Pure-Typst Workflow

`thyme` separates static-site orchestration from visual theming across two composable layers, driven exclusively by the standard `typst` CLI (`typst compile` and `typst watch`) without external wrapper CLIs or client-side JavaScript runtimes[1]:

[1] Typst `0.15`'s `typst watch -f bundle` includes a built-in live-reloading HTTP server (`--port`, `--no-reload`, `--no-serve`).

LAYER	IMPORT	RESPONSIBILITIES
Core SSG	`#import "@preview/thyme:0.1.0": *`	Seven names: `site`, `webpage`, `mount(path(...))`, `file`, `backlinks`, `#tr(...)` zero-JS CSS i18n + `hreflang` alternates, and `robots-txt`. Markup beyond content uses Typst's own `html.elem`. `sitemap.xml`, OpenGraph/Scholar `<meta>`, and summary or full-HTML `rss.xml` / `atom.xml` are emitted for you.
`blog` Preset	`#import blog: *` / `blog.site`	Opinionated scholarly monochrome blog (`oklch` / `light-dark()` + 2-column `@media print`), *STIX Two* + *Helvetica Neue* + *Commit Mono*, `@preview/marge` sidenotes + `#mcite` margin citations, static `/tags/` archives and post navigation, `#toc()`, Obsidian `<dl>` frontmatter, theorem/proof boxes, `zebraw`, `tablem`, and `cheq`.

02 Tadi Web `.typ` Sources & `sws.toml` Headers

When pages are mounted via Typst capability paths (`src: path("site.typ")`, `#mount(path("contact.typ"))`, `#post(path("posts/foo.typ"))`), `thyme` reads and co-emits

the verbatim `.typ` source alongside each HTML page (`site.typ`, `contact.typ`, `posts/foo.typ`) and links it via `.typ` in the top utility bar. Configure your static server (such as `static-web-server`) so browsers render `.typ` inline as UTF-8 plaintext:

```toml
[[advanced.headers]]
source = "**/*.typ"
headers = { Content-Type = "text/plain; charset=utf-8", Content-Disposition =
"inline" }
```

03 Public Surface

Two small re-export lists are the public API. Everything a site file writes by hand is in one of them; everything else in `src/` is a module-level function that the engine and the presets call.

3.1 Core — `#import "@preview/thyme:0.1.0": *`

NAME	PURPOSE	GROUP
`site`	Root site bundler: `#show: site.with(title:, url:, ...)`.	Build
`webpage`	A mounted sub-page document: `#show: webpage.with(...)`.	Build
`mount`	Mount a `.typ` file, a directory, or a literal body into the bundle.	Build
`file`	Co-emit a static file at a bundle route.	Build
`backlinks`	Compile-time backlinks to the pages that link here.	Build
`tr`	Declare text once per language: `tr(en: [...], de: [...])`.	i18n
`robots-txt`	Build a `robots.txt` string, as a starting point you then own.	SEO

Seven names. Everything a `site` or `webpage` applies for you is absent on purpose:

- Inline MathML is normalised for HTML export.
- Markup beyond plain content uses Typst's own `html.elem`.
- `sitemap.xml`, `rss.xml` and `atom.xml` are emitted by `site`. A `robots.txt` or `llms.txt` is copied byte for byte when you pass `path(...)`.
- Per-page assets go in the page's `assets:` parameter.

3.2 `blog` — `#import "@preview/thyme:0.1.0": blog` then `#import blog: *`

NAME	PURPOSE	GROUP
`site`	The blog bundler: `#show: blog.site.with(...)`.	Engine
`webpage`	A mounted sub-page document.	Engine
`mount`	Mount a `.typ` file, a directory, or a literal body.	Engine
`file`	Co-emit a static file at a bundle route.	Engine
`backlinks`	Compile-time backlinks.	Engine
`post`	One post, with its route, feed entry and metadata.	Posts
`posts`	Posts with dates, descriptions and reading times.	Posts
`theme-default`	The palette dictionary, for reading a colour or font in your markup.	Theme
`sidenote`	Numbered note in the right margin.	Notes
`sn`	Short form of `sidenote`.	Notes
`mn`	Margin note without a number.	Notes
`mcite`	Bibliography citation placed in the margin.	Notes
`callout`	Boxed aside.	Blocks
`sbox`	Generic boxed block.	Blocks

NAME	PURPOSE	GROUP
`definition`	Academic environment.	Blocks
`theorem`	Academic environment.	Blocks
`lemma`	Academic environment.	Blocks
`proposition`	Academic environment.	Blocks
`corollary`	Academic environment.	Blocks
`example`	Academic environment.	Blocks
`recipe`	Academic environment.	Blocks
`remark`	Academic environment.	Blocks
`proof`	Academic environment.	Blocks
`note-box`	Boxed aside.	Blocks
`tip-box`	Boxed aside.	Blocks
`warning-box`	Boxed aside.	Blocks
`important-box`	Boxed aside.	Blocks
`epigraph`	Opening quotation.	Blocks
`codeblock`	Fenced code with a copy button.	Blocks
`diagram`	Captioned figure for a diagram.	Blocks
`wide`	Break out of the text column.	Blocks
`kbd`	Keyboard key.	Blocks
`hr`	Section divider.	Blocks
`links`	Link row: `links(direction: "row", ...)`.	Blocks
`toc`	Table of contents for the current document.	Blocks
`tablem`	Markdown-style table.	Tables
`three-line-table`	Booktabs-style table.	Tables
`grid-table`	Grid table.	Tables
`zebraw`	Striped table rows.	Tables

Applied for you by `blog.site`, and absent on purpose:

- `blog.site(post-nav: true)` puts newer/older links under every post.
- `blog.site` builds the `/tags/` index and the per-tag archives.
- A post's `assets: ((name, bytes),)` parameter emits a co-located asset.

`checklist`, `showybox`, `zebraw-init` and `zebraw-themes` are `@preview` packages that `blog` wraps; import them from their own package when you need them directly.

NAME	PURPOSE	GROUP
`site`	The blog bundler: `#show: blog.site.with(...)`.	Engine
`webpage`	A mounted sub-page document.	Engine
`mount`	Mount a `.typ` file, a directory, or a literal body.	Engine
`file`	Co-emit a static file at a bundle route.	Engine
`backlinks`	Compile-time backlinks.	Engine
`post`	One post, with its route, feed entry and metadata.	Posts
`posts`	Posts with dates, descriptions and reading times.	Posts
`theme-default`	The palette dictionary, for reading a colour or font in your markup.	Theme
`sidenote`	Numbered note in the right margin.	Notes
`sn`	Short form of `sidenote`.	Notes
`mn`	Margin note without a number.	Notes
`mcite`	Bibliography citation placed in the margin.	Notes
`callout`	Boxed aside.	Blocks

NAME	PURPOSE	GROUP
`sbox`	Generic boxed block.	Blocks
`definition`	Academic environment.	Blocks
`theorem`	Academic environment.	Blocks
`lemma`	Academic environment.	Blocks
`proposition`	Academic environment.	Blocks
`corollary`	Academic environment.	Blocks
`example`	Academic environment.	Blocks
`recipe`	Academic environment.	Blocks
`remark`	Academic environment.	Blocks
`proof`	Academic environment.	Blocks
`note-box`	Boxed aside.	Blocks
`tip-box`	Boxed aside.	Blocks
`warning-box`	Boxed aside.	Blocks
`important-box`	Boxed aside.	Blocks
`epigraph`	Opening quotation.	Blocks
`codeblock`	Fenced code with a copy button.	Blocks
`diagram`	Captioned figure for a diagram.	Blocks
`wide`	Break out of the text column.	Blocks
`kbd`	Keyboard key.	Blocks
`hr`	Section divider.	Blocks
`links`	Link row: `links(direction: "row", ...)`.	Blocks
`toc`	Table of contents for the current document.	Blocks
`tablem`	Markdown-style table.	Tables
`three-line-table`	Booktabs-style table.	Tables
`grid-table`	Grid table.	Tables
`zebraw`	Striped table rows.	Tables

The preset applies `with-html-math` itself, so a post does not need to. `checklist`, `showybox`, `zebraw-init` and `zebraw-themes` are `@preview` packages that `blog` wraps; import them from their own package when you need them directly.

04 Internal Modules (`src/**/*.typ`)

These modules build a site. They are documented for anyone extending `thyme` or reading `site`'s behaviour, and their import paths stay stable. The names inside them do not: they carry no compatibility promise and change whenever the engine or a preset needs them to.

4.1 Monochrome Theme & Fonts (`src/blog/theme.typ`)

`css-hex``(col) ->` `str`

Format any Typst `color` as a lowercase `#rrggbb` hex string, for contexts that need a plain sRGB literal rather than `oklch(...)` (e.g. `<meta name="theme-color">`).

`resolve-theme``(overrides) ->` `dictionary`

Merge a user `theme` override dictionary onto `theme-default`.

PARAMETERS.

`overrides` `dictionary` `none`
Partial theme dictionary overriding any subset of `theme-default` keys.

`active-theme``()` -> `dictionary`

Resolve the active theme inside `context` (preferring any enclosing per-post `<blog-post-theme>` override over the site-wide `<site-config>`).

`stylesheet-css``(theme: theme-default)` -> `str`

Render `src/blog/style.css` substituting all `oklch(...)` palette tokens, `--serif` / `--math` / `--sans` / `--mono` font stacks, and `@font-face` declarations from `theme`.

PARAMETERS.

`theme` `dictionary` `= theme-default`
Theme dictionary (`theme-default` or user overrides).

`default-woff2-files` `array`

Canonical `.woff2` filenames expected under `theme.font-base` when self-hosting upstream *STIX Two* (`v2.13b171`) and *Commit Mono* (`v1.143`) web fonts.

`theme-default` `dictionary`

Default perceptual `oklch(L, 0, 0deg)` monochrome palette, unified *STIX Two* + *Helvetica Neue* + *Commit Mono* font stacks, and paged preview geometry.